

```
bytes :: B.ByteString
bytes = "Hello, World" :: B.ByteString
```

```
main :: IO (Either Reply B.ByteString)
main = do
  conn <- connect defaultConnectInfo
  runRedis conn $ echo bytes
```

Loading the above code in GHCi produces the following output:

```
ghci> main
Right "Hello, World"
```

The type signature of the `echo` function is as follows:

```
echo
  :: RedisCtx m f =>
     Data.ByteString.Internal.ByteString
  -> m (f Data.ByteString.Internal.ByteString)
```

You can set a value to a key using the `set` function in *hedis*. An example is shown below:

```
{-# LANGUAGE OverloadedStrings #-}
import Database.Redis

main :: IO (Either Reply Status)
main = do
  conn <- connect defaultConnectInfo
  runRedis conn $ set "a" "apple"
```

Loading the above `set.hs` code in GHCi and testing the same produces the following output:

```
ghci> :l set.hs
[1 of 1] Compiling Main             ( set.hs, interpreted )
Ok, modules loaded: Main.

ghci> main
Right Ok
```

The type signature of the `set` function is shown below:

```
ghci> :t set
set
  :: RedisCtx m f =>
     Data.ByteString.Internal.ByteString
  -> Data.ByteString.Internal.ByteString -> m (f Status)
```

You can verify the value of the key 'a' from the *redis-cli* command, and it must return the value "apple":

```
127.0.0.1:6379> get a
"apple"
```

You can also retrieve the value of a key using the `get` function. For example:

```
{-# LANGUAGE OverloadedStrings #-}
import Database.Redis
import Control.Monad.IO.Class

main :: IO ()
main = do
  conn <- connect defaultConnectInfo
  runRedis conn $ do
    result <- get "a"
    liftIO $ print result
```

Executing the above code in GHCi gives the expected result:

```
ghci> :l get.hs
[1 of 1] Compiling Main             ( get.hs, interpreted )
Ok, modules loaded: Main.

ghci> main
Right (Just "apple")
```

The `liftIO` function transforms an *IO* action into a *Monad*. Its type signature is shown below:

```
ghci> :t liftIO
liftIO :: MonadIO m => IO a -> m a
The type signature of the get function is as follows:
ghci> :t get
get
  :: RedisCtx m f =>
     Data.ByteString.Internal.ByteString
  -> m (f (Maybe Data.ByteString.Internal.ByteString))
```

You are encouraged to read the *Database.Redis* documentation page that contains a comprehensive list of commands and their usage at <https://hackage.haskell.org/package/hedis-0.6.5/docs/Database-Redis.html>.

Accessing the PostgreSQL database

We shall now explore accessing a PostgreSQL database using the *postgresql-simple* (0.4.10.0) package. You will need to install and configure PostgreSQL for your GNU/Linux distribution. Please follow your distribution documentation to do so. On Fedora, for example, you can install the database server using the following command:

```
$ sudo yum install postgresql-server postgresql-contrib
```

You can then start the database server using the following service command: